Short Answer (Questions 1–3)

1. Answer: Two approaches — with trade-offs:

- Option 1 — Separate history table: Create PRICE_HISTORY(ProdID, Price, EffectiveDate). Every price change adds a new row. Full audit trail preserved. Trade-off: queries for the current price are slightly more complex (need to find the most recent EffectiveDate).
- Option 2 — Associative entity at time of transaction: Store Price_Per_Item in ORDER_LINE at the moment of purchase. Captures the price as it was when ordered. Trade-off: this only preserves historical prices tied to actual orders — it does not give you a complete standalone price history independent of sales.

---

2. Answer: Associative entity (ENROLLMENT). Justification:

Grade and Semester describe the specific pairing of a student and a course — not the student alone, and not the course alone. The same student receives different grades in different courses, and the same course gives different grades to different students. Since the value depends on both entities together, it belongs on the ENROLLMENT associative entity: ENROLLMENT(StudentID FK, CourseID FK, Grade, Semester).

---

3. Answer: A unary relationship is a relationship where an entity is related to itself — also called a recursive or self-referencing relationship.

Model: Entity = EMPLOYEE. Relationship = SUPERVISES (unary, drawn as a loop on EMPLOYEE). Cardinality = 1:N — one employee (supervisor) may supervise many employees, but each employee has at most one direct supervisor. In the relational schema this maps to a self-referencing FK: EMPLOYEE(EmpID, EmpName, ..., ManagerID) where ManagerID references EmpID in the same table. ManagerID is nullable because not every employee has a supervisor.

True / False (Questions 4–8)

4. Answer: False.

A table that stores multiple phone numbers in a single cell (e.g., '215-555-0101, 610-555-0189') violates atomicity — every attribute value in a relation must be a single, indivisible value. Storing a list in one cell means the column is not atomic, so the table does not qualify as a relation.

---

5. Answer: True.

Declaring a PRIMARY KEY in Oracle automatically enforces both uniqueness and NOT NULL on that column. A primary key constraint implies both constraints — you do not need to add them separately.

---

6. Answer: False.

A foreign key column does NOT have to be NOT NULL. Whether it requires a value depends on the business rule. If the relationship is optional (partial participation), the FK may be NULL — meaning that instance is not currently associated with a parent. For example, EMPLOYEE.ManagerID is nullable because not every employee has a manager. NOT NULL is only added when participation is mandatory.

---

7. Answer: False.

Two rows in a relation cannot share the same primary key value under any circumstances. The primary key must uniquely identify every row — this uniqueness guarantee holds regardless of what other columns contain. Rows with the same PK are duplicates by definition and violate entity integrity.

---

8. Answer: False.

Grade belongs on the ENROLLMENT associative entity, not on STUDENT. Grade describes a specific (student, course) pairing — it only makes sense when you know both which student and which course are involved. Placing Grade on STUDENT implies each student has only one grade for all courses, which is logically incorrect.

Multiple Choice (Questions 9–15)

9. Answer: C.

A foreign key value of 50 when no row with PK = 50 exists in the parent table directly violates referential integrity — the FK must match an existing PK or be NULL. Option A is valid (NULL is allowed in optional FKs). Option B is expected and normal (multiple child rows sharing the same FK is the entire point of a 1:N relationship). Option D is not a referential integrity issue.

---

10. Answer: C — Modification anomaly.

CustomerCity depends on CustomerID, not on OrderID (the PK). One real-world fact — the customer's city — is stored across many rows. When it changes, every row must be updated. Failing to update even one row leaves the data inconsistent. This is the defining characteristic of a modification anomaly.

---

11. Answer: B.

Entity integrity requires that every relation has a primary key, and that PK values are always unique and never NULL. Option A describes referential integrity. Option C is too strict — NULLs are permitted in non-key columns. Option D describes CHECK constraint enforcement.

---

12. Answer: B.

This is a unary (recursive) relationship — EMPLOYEE relates to itself. In the relational schema, this maps to a self-referencing FK: a ManagerID column in EMPLOYEE that references EmpID in the same table. Option A is incorrect because you would not create two separate EMPLOYEE entities. Option C would be used if the relationship had its own attributes. Option D would be used if managers had entirely different attributes from regular employees.

---

13. Answer: C.

An associative entity is needed when the M:N relationship has its own attributes (like Grade, DateCompleted, Quantity) or when the association has independent meaning that needs to be tracked on its own. Option A is too strong — a plain M:N can be left as-is if no attributes are needed and it has no independent meaning. Options B and D are incorrect.

---

14. Answer: D — Total specialization + Disjoint.

'Every account must be one of the three' → Total specialization (every supertype instance must belong to at least one subtype — double line). 'Never more than one at the same time' → Disjoint rule (an instance belongs to exactly one subtype — 'd' in the circle). Both constraints must apply.

15. Answer: D — Partial specialization + Overlap.

'Some people are neither' → Partial specialization (a PERSON does not have to belong to any subtype — single line). 'Can be both simultaneously' → Overlap rule (an instance can belong to more than one subtype — 'o' in the circle).

Read the Schema — Course Enrollment (Questions 16–18)

Schema for reference:

```
STUDENT   ( StudentID, LastName, FirstName, Email, MajorName, MajorBuilding )
COURSE    ( CourseID, CourseName, Credits, DeptCode, ProfessorID )
PROFESSOR ( ProfessorID, ProfName, DeptCode )
SECTION   ( SectionID, CourseID, Semester, Year, Enrollment )
ENROLLS   ( StudentID, CourseID, Grade, EnrollDate )
underline = Primary Key    dashed underline = Foreign Key
```

16. Answer: ENROLLS resolves a M:N relationship between STUDENT and COURSE.

Evidence: its primary key is composite — (StudentID, CourseID) — made up of FK references to both participating entities. This is the standard pattern for a junction/associative table. The presence of Grade and EnrollDate as additional columns confirms it is an associative entity with its own attributes describing the specific enrollment pairing.

17. Answer: Yes — the professor can be inserted. No insertion anomaly exists here.

PROFESSOR is its own independent table. A new professor row can be inserted without needing any COURSE row to exist at the same time. This is correct design — having a dedicated PROFESSOR table prevents the insertion anomaly. If professor data were stored only inside the COURSE table, you could not record a professor until they were assigned to teach a course, which would be a classic insertion anomaly.

18. Answer: ProfessorID in COURSE enforces referential integrity — it ensures every course is taught by a professor who actually exists in the PROFESSOR table.

If someone tries to INSERT a COURSE row with a ProfessorID value that does not exist in PROFESSOR, Oracle will reject the insert and raise ORA-02291 (integrity constraint violated — parent key not found). The insert fails entirely — no partial inserts occur.

Read the Schema — Hospital System (Questions 19–21)

Schema for reference:

```
PATIENT    ( MRN, PatientName, DOB, PhysicianID )
PHYSICIAN  ( PhysicianID, PhysicianName, Specialty )
WARD       ( WardID, WardName, HeadNurseID )
NURSE      ( NurseID, NurseName, WardID NOT NULL )
ADMISSION  ( AdmissionID, MRN, WardID, AdmitDate, DischargeDate )
underline = Primary Key    dashed underline = Foreign Key    NOT NULL where shown
```

19. Answer: NURSE.WardID being NOT NULL means every nurse must be assigned to a ward — the relationship is mandatory on the NURSE side (total participation).

A nurse row cannot be inserted without providing a valid WardID. This enforces the business rule that nurses always belong to a ward. In a Crow's Foot diagram this is shown as a mandatory line marker (|) on the NURSE end of the NURSE–WARD relationship.

---

20. Answer: PATIENT.PhysicianID being nullable (no NOT NULL) means the relationship between PATIENT and PHYSICIAN is optional — a patient does not have to have a physician on record.

This is an intentional modeling decision, not a flaw. A patient may be admitted before a physician is assigned, or the hospital may allow patients who are treated by multiple physicians (in which case a separate TREATMENT associative entity would be even better). The nullable FK represents partial participation on the PATIENT side.

---

21. Answer: Yes — the ADMISSION table can record the same patient being admitted more than once.

ADMISSION uses its own surrogate primary key (AdmissionID), so multiple rows can exist with the same MRN value. Each row represents a distinct admission event with its own AdmitDate and DischargeDate. If MRN were the PK of ADMISSION, only one admission per patient would ever be possible — the surrogate key design correctly supports a full admission history.

Read the Schema — Normalization (Questions 22–24)

Schema for reference:

```
ORDER_RECORD ( OrderID, CustomerID, CustomerName, CustomerEmail,
               ProductID, ProductName, ProductCategory,
               Quantity, UnitPrice, OrderDate )
underline = Primary Key   (single unnormalized table, no FKs)
```

22. Answer: Two groups of partial dependencies exist (columns that depend on part of the PK, not OrderID alone):

Note: With OrderID as the only PK column, technically all non-key attributes are 'fully dependent' on it in a strict 2NF sense — since there is no composite key, no partial dependency can exist by definition. However, the question is testing whether you can identify the logical groupings of data that do not belong together. The intended answer is:

- CustomerID → CustomerName, CustomerEmail (customer data depends on CustomerID, not on OrderID — this is a transitive dependency via CustomerID)
- ProductID → ProductName, ProductCategory (product data depends on ProductID, not on OrderID — also a transitive dependency via ProductID)

These would be partial dependencies if the PK were composite (e.g., OrderID + ProductID). Either way, they represent data that logically belongs in separate tables.

---

23. Answer: C — Insertion anomaly.

You cannot add a new product to the catalog without also providing an OrderID — but no orders exist yet for that product. Since ORDER_RECORD is the only table and has no separate PRODUCT table, there is nowhere to record a product that has not yet been ordered. This is a textbook insertion anomaly: the table structure prevents inserting data that should be able to exist independently.

---

24. Answer: 3NF decomposition — four tables:

```
CUSTOMER  ( CustomerID,  CustomerName, CustomerEmail )
           PK: CustomerID

PRODUCT   ( ProductID,   ProductName,  ProductCategory )
           PK: ProductID

ORDER_HDR ( OrderID,     CustomerID,   OrderDate )
           PK: OrderID     FK: CustomerID → CUSTOMER

ORDER_LINE( OrderID,     ProductID,    Quantity, UnitPrice )
           PK: (OrderID, ProductID)    FK: OrderID → ORDER_HDR
                                       FK: ProductID → PRODUCT
```

Note: UnitPrice stays in ORDER_LINE (not PRODUCT) because it captures the price at the time of that specific order, which may differ from the current product price.

Can You Insert? (Questions 25–29)

Schema for reference:

```
DEPARTMENT ( DeptID, DeptName, ManagerID NULL )
EMPLOYEE   ( EmpID, EmpName, DeptID NOT NULL, Salary, ManagerID NULL )
PROJECT    ( ProjectID, ProjectName, DeptID NOT NULL )
WORKS_ON   ( EmpID, ProjectID, HoursPerWeek )   PK: (EmpID, ProjectID)
EMPLOYEE.DeptID   → DEPARTMENT(DeptID)
EMPLOYEE.ManagerID → EMPLOYEE(EmpID)  (self-referencing, nullable)
PROJECT.DeptID    → DEPARTMENT(DeptID)
WORKS_ON.EmpID    → EMPLOYEE(EmpID)
WORKS_ON.ProjectID → PROJECT(ProjectID)
DEPARTMENT.ManagerID → EMPLOYEE(EmpID)  (nullable)
```

25. Answer: B.

The insert fails. EMPLOYEE.DeptID is NOT NULL and is a FK referencing DEPARTMENT(DeptID). DeptID 99 does not exist in DEPARTMENT, so the FK check fails. Oracle raises ORA-02291 (integrity constraint violated — parent key not found). The NOT NULL constraint is also violated simultaneously, but the FK violation is the primary reason cited.

---

26. Answer: B.

The insert fails. EMPLOYEE.DeptID is explicitly defined as NOT NULL. Passing NULL for DeptID violates this constraint. Oracle raises ORA-01400 (cannot insert NULL into a NOT NULL column). This reflects mandatory participation — every employee must belong to a department.

---

27. Answer: B.

The insert fails. DEPARTMENT.ManagerID is a FK referencing EMPLOYEE(EmpID). Even though ManagerID is nullable in general, the value 202 is provided — and EmpID 202 does not yet exist in EMPLOYEE. A non-NULL FK value must match an existing PK. Oracle raises ORA-02291.

Resolution of the chicken-and-egg problem: (1) INSERT DEPARTMENT with ManagerID = NULL first, (2) INSERT the EMPLOYEE with DeptID pointing to that department, (3) then UPDATE DEPARTMENT SET ManagerID = 202.

---

28. Answer: B.

The insert fails. WORKS_ON.ProjectID is a FK referencing PROJECT(ProjectID). ProjectID 50 does not exist in PROJECT. Even though EmpID 201 is valid, both FK columns in a junction table must reference valid parent rows. Oracle raises ORA-02291.

---

29. Answer: B.

The insert fails. The composite PK of WORKS_ON is (EmpID, ProjectID). A row with the combination (201, 30) already exists. Inserting a duplicate primary key violates entity integrity — the uniqueness requirement of the PK. Oracle raises ORA-00001 (unique constraint violated). Both FK values may be perfectly valid; the issue is the duplicate primary key, not referential integrity.

Read the SQL (Questions 30–34)

30. Answer: Creates the ENROLLMENT table with a 3-column composite primary key and four named constraints.

Constraints created:

- NOT NULL on StudentID, CourseID, Semester — none of these three columns can be empty. They form the composite PK so none can be NULL.
- CHECK on Grade — only the values 'A', 'B', 'C', 'D', 'F', 'W', or 'I' are accepted. Any other value is rejected.
- DEFAULT SYSDATE on EnrollDate — if no date is provided on INSERT, Oracle automatically uses the current date and time.
- pk_enroll PRIMARY KEY (StudentID, CourseID, Semester) — composite PK. A student can be enrolled in the same course across multiple semesters, but not twice in the same semester.
- fk_student FOREIGN KEY (StudentID) → STUDENT(StudentID) — enforces that every StudentID in ENROLLMENT references an existing student.
- fk_course FOREIGN KEY (CourseID) → COURSE(CourseID) — enforces that every CourseID references an existing course.

Note: the study guide's SQL is missing a comma after the fk_student REFERENCES clause — as written, it would produce a syntax error. The intended behavior is as described above.

---

31. Answer: Adds a named CHECK constraint to EMPLOYEE requiring Salary to be greater than 0 and less than 500,000.

If any existing row in EMPLOYEE already has Salary = 0 (or any value outside the range), Oracle will reject the entire ALTER TABLE statement with ORA-02293 (cannot validate constraint — check constraint violated). The constraint is not added at all — all existing data must already satisfy the constraint before it can be applied. If all existing rows pass, the constraint is added and will enforce the range on all future INSERTs and UPDATEs.

---

32. Answer: Returns one row per department showing the number of qualifying employees and their average salary, filtered and sorted.

Clause by clause:

- FROM DEPARTMENT D, EMPLOYEE E — joins both tables (implicit inner join).

- WHERE D.DeptID = E.DeptID — the join condition; keeps only matching rows. AND E.Salary > 50000 — further filters to only employees earning more than $50,000.
- GROUP BY D.DeptName — groups the remaining rows by department name.
- COUNT(E.EmpID) AS NumEmployees — counts how many qualifying employees are in each department group.
- ROUND(AVG(E.Salary),2) AS AvgSalary — calculates the average salary for the group, rounded to 2 decimal places.
- HAVING COUNT(E.EmpID) > 3 — keeps only departments that have MORE THAN 3 employees earning over $50k.
- ORDER BY AvgSalary DESC — sorts the final result from highest to lowest average salary.

Final result: a list of department names with employee counts and average salaries, limited to departments with more than 3 high-earning employees, in descending order of average salary.

---

33. Answer: No — it will not work. It will always return zero rows even if NULL values exist in ManagerID.

In SQL, NULL represents the absence of a known value. The expression ManagerID = NULL evaluates to UNKNOWN (not TRUE or FALSE) for every single row — so no rows ever pass the WHERE filter. You cannot use = to test for NULL.

Corrected query:

```
SELECT EmpName
FROM   EMPLOYEE
WHERE  ManagerID IS NULL;
```

IS NULL is the only correct operator for detecting NULL values in Oracle SQL.

---

34. Answer: The three statements add a new nullable FK column, a referential integrity constraint with ON DELETE SET NULL, and an email format CHECK constraint.

End state of STUDENT after all three execute:

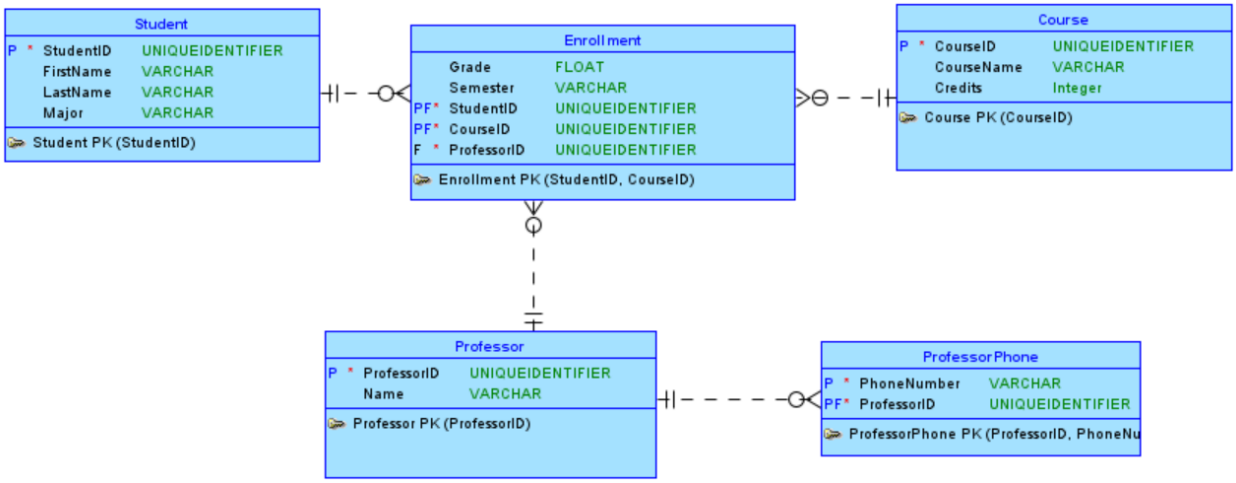
- AdvisorID column added: NUMBER(8), nullable by default (no NOT NULL specified). Every existing row gets AdvisorID = NULL.
- fk_advisor constraint added: AdvisorID must match an existing ProfessorID in PROFESSOR, or be NULL. ON DELETE SET NULL means: if a professor is deleted, all students whose AdvisorID pointed to that professor have their AdvisorID automatically set to NULL — students are not deleted.
- chk_email constraint added: every value in the Email column must contain the '@' character. Any INSERT or UPDATE that provides an email without '@' is rejected.

ER Design Solutions (Questions 35–37)

The following describe what a complete, correct diagram must include. Drawings will vary — evaluate against these criteria.

35. University Course Registration

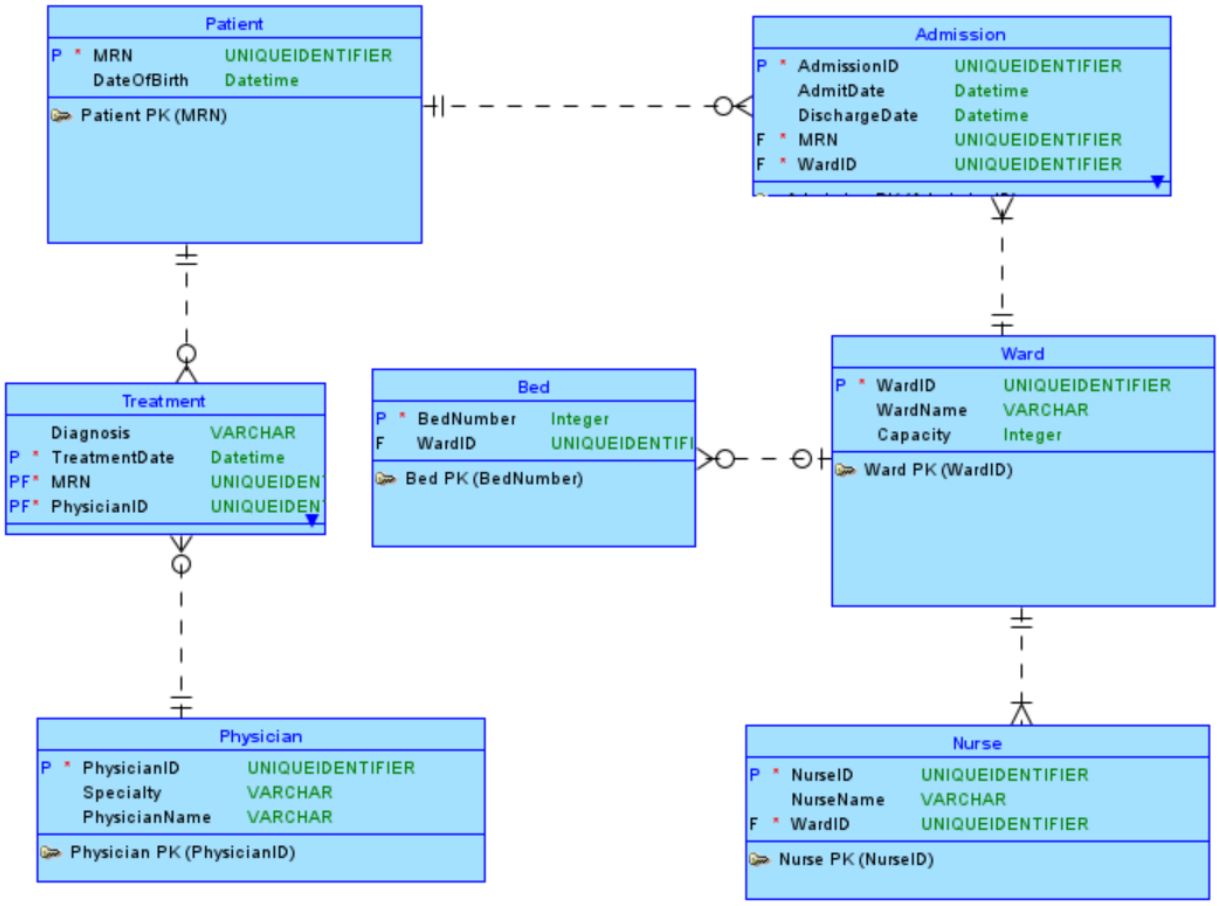
*PhoneNumber is multivalued — maps to a separate PROF_PHONE entity



COMMON ERRORS: Placing Grade on STUDENT or COURSE
Missing mandatory marker on PROFESSOR end of COURSE relationship
Not creating ENROLLMENT as an associative entity

36. Hospital System

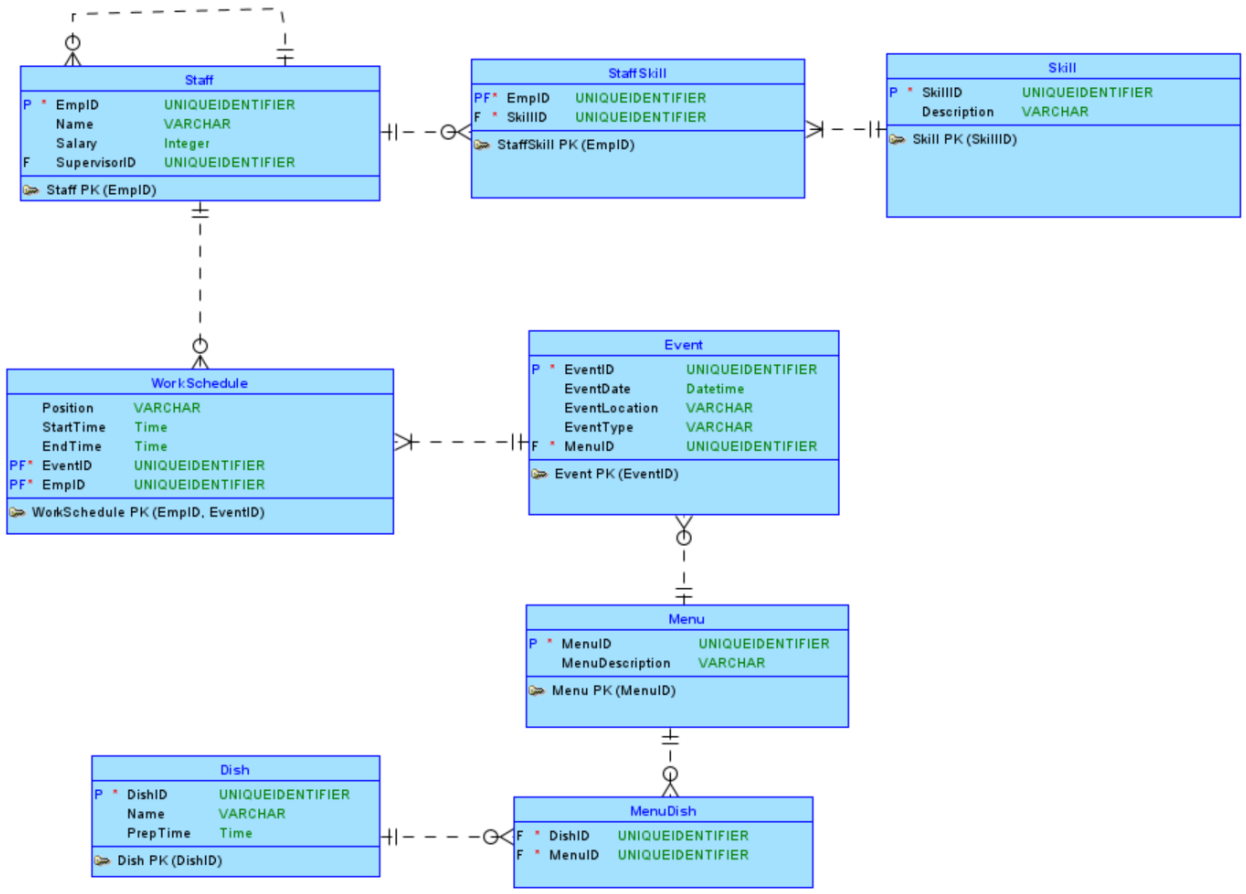
ENTITIES & ATTRIBUTES
PATIENT: MRN (PK), PatientName, DateOfBirth
WARD: WardID (PK), WardName, Capacity
BED: BedNumber (partial key), WardID (FK) | Depends on WARD — shown by mandatory line end
NURSE: NurseID (PK), NurseName, WardID (FK NOT NULL)
PHYSICIAN: PhysicianID (PK), PhysicianName, Specialty
ADMISSION (junction): AdmissionID (PK), MRN (FK), WardID (FK), AdmitDate, DischargeDate
TREATMENT (junction): PhysicianID (FK), MRN (FK), Diagnosis, TreatmentDate | PK = (PhysicianID, MRN, TreatmentDate)



COMMON ERRORS: Storing Age as an attribute
Not creating TREATMENT entity, placing Diagnosis on PHYSICIAN or PATIENT
Not showing mandatory participation on NURSE–WARD .

37. Catering & Events

ENTITIES & ATTRIBUTES
STAFF: EmpID (PK), Name, Salary, SupervisorID (FK, self-referencing, nullable)
STAFF_SKILL: EmpID (FK), Skill | PK = (EmpID, Skill) — Skills is multivalued, needs its own entity
EVENT: EventID (PK), EventDate, EventLocation, EventType
MENU: MenuID (PK), MenuDescription
DISH: DishID (PK), DishName, PrepTime
WORK_SCHEDULE (junction): EmpID (FK), EventID (FK), Position, StartTime, EndTime | PK = (EmpID, EventID)
MENU_DISH (junction): MenuID (FK), DishID (FK) | PK = (MenuID, DishID)



COMMON ERRORS: Not creating STAFF_SKILL for the multivalued Skills attribute .
Missing the self-referencing loop for SUPERVISES.
Not making WORK_SCHEDULE an associative entity with attributes